Hvilke tests skal I lave?

Konkrete testforslag til jeres C#-backend, MySQL-database og JavaScript-frontend. Alle krav i den indsatte opgavetekst er medtaget.

Testtype og designteknik er to forskellige ting

- Unit test: tester én klasse eller regel isoleret. Brug xUnit og simple fakes til repositories og generatorer. Ingen rigtig database eller HTTP.
- Integrationstest: tester samarbejdet mellem komponenter, fx repository + MySQL eller controller + service + JSON via WebApplicationFactory.
- API-test: sender HTTP-requests med Postman og kontrollerer svar med scripts. Kør den eksporterede samling i CI med Newman.
- End-to-end-test (E2E): bruger Playwright til at betjene den rigtige frontend gennem API til en separat testdatabase.

Sådan opfylder I designkravene

- Black-box (BB): udled testcases manuelt fra krav. Brug ækvivalensklasser, grænseværdier og en beslutningstabel for køn og CPR-paritet.
- White-box (WB): brug Coverlet til at måle linje- og grendækning. Find udækkede kodegrene, tilføj målrettede tests og mål igen. Se side 6.
- Erfaringsbaseret (ER): undersøg danske tegn, foranstillede nuller, skudår, databasefejl og gentagne klik. Gem fund som regressionstests.

Foreslået placering og prioritet

C#-tests: tests/PersonalData.UnitTests, tests/PersonalData.IntegrationTests og tests/PersonalData.E2ETests. HTTP-samling: tests/Postman. CI: .github/workflows/tests.yml.

Start her: CPR og konsistens, antalgrænser, danske tegn og én rigtig E2E-test. Udbyg derefter med resten. Testene nedenfor er en plan, ikke allerede udførte tests.

Unit tests: generatorer

Forslag til xUnit-tests. Brug fast tid og kontrolleret Random, så testene kan gentages og ikke afhænger af held.

- U01 - Navn og køn kommer fra samme person
  Unit | BB: ækvivalensklasser. Et fake repository med to forskellige personer returnerer altid en hel post; fornavn, efternavn og køn må ikke blandes.
- U02 - Tom navneliste
  Unit | WB + ER. Et tomt IPersonNameRepository giver InvalidOperationException. Med én person returneres denne person.
- U03 - Fødselsdatoens to grænser
  Unit | BB: grænseværdier. Fastlås dags dato. Tving tidligste og seneste mulige dag: begge er inkluderet, og ingen dato ligger i fremtiden.
- U04 - Skudår og årsskifte
  Unit | ER. Tving 29/02/2024 samt 31/12 og 01/01. Forvent gyldige DateOnly-værdier. Afklar først startår 1900 kontra de seneste 100 år.
- U05 - CPR-format og foranstillede nuller
  Unit | BB + ER. Født 06/01/2005 giver præfikset 060105. Nummeret har præcis 10 cifre. Tving de tre næste cifre til 000 og 999.
- U06 - CPR følger kønnet
  Unit | BB: beslutningstabel + WB. Female giver sidste ciffer 0, 2, 4, 6 eller 8; Male giver 1, 3, 5, 7 eller 9. Tving yderpunkterne for begge køn.
- U07 - Ukendt køn
  Unit | WB + ER. Send fx (Gender)99 til CprGenerator. Forvent ArgumentOutOfRangeException.
- U08 - Telefonnummerets præfiks og længde
  Unit | BB + WB. Tving præfikser med 1, 2 og 3 cifre. Resultatet har præcis 8 cifre og begynder med et tilladt præfiks. Tving også 0 og 9 i resten.
- U09 - Vejnavn og husnummer
  Unit | BB + WB. Vejnavnet har 40 tegn, begynder med et bogstav og indeholder kun tilladte bogstaver/mellemrum. Test husnummer 1 og 999 med og uden A/Z.
- U10 - Etage og alle dørtyper
  Unit | BB + WB. Test st, etage 1 og 99 samt th, tv, mf, tal 1/50 og bogstav med/uden bindestreg og tal 1/999. Se de konkrete grene på side 6.
- U11 - Postnummer og by hænger sammen
  Unit | BB + ER. Brug fake postnummerrækker. Den valgte by og det valgte postnummer skal komme fra samme række.
- U12 - Tom postnummerliste
  Unit | WB + ER. Et tomt IPostalCodeRepository giver InvalidOperationException, før en adresse returneres.

Service og integration

Hold service-tests isolerede. Brug en separat testdatabase til integration; ændr ikke jeres almindelige udviklingsdata.

FakePersonService - isolerede tests

- U13 - En hel person er konsistent
  Unit | BB + ER. Fakes giver kendt navn, køn, dato, adresse og telefon. Kontrollér alle felter og at CPR-generatoren modtager præcis samme køn og dato.
- U14 - De delvise svar
  Unit | BB. Test GetNameAndGender, GetNameGenderAndBirthDate, GetCprNameAndGender, GetCprNameGenderAndBirthDate, GetCpr, GetAddress og GetPhoneNumber mod faste fake-værdier.
- U15 - Gyldigt antal personer
  Unit | BB: grænseværdier. GetPersons(2), (3), (99) og (100) returnerer præcis det ønskede antal komplette personer.
- U16 - Ugyldigt antal personer
  Unit | BB: ækvivalensklasser. GetPersons(-1), (0), (1) og (101) giver ArgumentOutOfRangeException uden at generere personer.

Repository og API - rigtige komponenter

- I01 - JSON-filen læses korrekt
  Integration | BB + ER. Læs en rigtig testfil. Kontrollér feltnavne, female/male, danske tegn og at gentagne GetAll-kald genbruger de indlæste data.
- I02 - JSON-fejl
  Integration | BB + ER. Afprøv tom persons-liste, manglende persons, ugyldig JSON og manglende fil. Forvent henholdsvis InvalidDataException, JsonException eller filfejl efter den aktuelle kontrakt.
- I03 - MySQL-rækker og Unicode
  Integration | BB + ER. Indsæt kendte testbyer som Vanløse, Rødovre og Århus. Kontrollér præcise værdier, firecifrede postnumre og de samme navne i API-svaret.
- I04 - Fejlsekvenser og NULL
  Integration | ER + WB. Indsæt fx VanlÃ¸se og rigtige æøåÆØÅ. Den nuværende læser korrigerer fejlsekvenser og bevarer korrekt tekst. Rækker med NULL-by filtreres fra.
- I05 - Konfiguration og databasefejl
  Integration | ER. Manglende Addresses-connection string giver InvalidOperationException. Tom tabel giver generatorfejl. En utilgængelig testdatabase må ikke give et falsk succes-svar.
- I06 - HTTP, routing og JSON
  Integration | BB. Brug WebApplicationFactory til alle ni routes: 200 ved gyldige kald, 400 ved ugyldigt count, gender som female/male og birthDate som yyyy-MM-dd.

Praktisk: WebApplicationFactory kan kræve public partial class Program. JSON-fejltests kræver en isoleret testfil eller et lille injicerbart filsti-parameter; overskriv ikke produktionsfilen.

API-tests i Postman

Opret en collection med {{baseUrl}} = http://localhost:5277. Tilføj scripts under Post-response/Tests og eksportér som collection v2.1 til Newman.

ID og GET-route	Kontrollér ud over HTTP 200 og JSON
A01 /api/cpr	cpr er en streng med præcis 10 cifre.
A02 /api/name-gender	firstName/lastName er udfyldt; gender er female eller male.
A03 /api/name-gender-dob	Som A02 samt gyldig birthDate i yyyy-MM-dd, ikke i fremtiden.
A04 /api/cpr-name-gender	CPR-paritet passer til gender; navne er udfyldt.
A05 /api/cpr-name-gender-dob	Som A04; de første seks CPR-cifre matcher birthDate.
A06 /api/address	Adressefelter findes; postnummer/by findes som par i testdatasættet.
A07 /api/phone	phoneNumber er en streng med 8 cifre og et tilladt præfiks.
A08 /api/person	Alle felter, adresse og telefon findes; CPR, dato og køn passer sammen.
A09 /api/persons?count=2	Et array med præcis count komplette personer; gentag med 3, 99 og 100.

- A10 - Ugyldigt count

  API | BB: grænseværdier/klasser. Afprøv -1, 0, 1, 101, abc, 2.5 samt manglende count. Forvent HTTP 400. Assertér også fejlformat uden at kræve frameworkets præcise fejltekst.

Eksempel på tests til /api/person og /api/cpr-name-gender-dob:

```
pm.test("HTTP 200", () => pm.response.to.have.status(200));
const p = pm.response.json();
pm.test("CPR passer til dato og køn", () => {
  pm.expect(p.cpr).to.match(/^\d{10}$/);
  const [y, m, d] = p.birthDate.split("-");
  pm.expect(p.cpr.slice(0, 6)).to.eql(d + m + y.slice(-2));
  pm.expect(Number(p.cpr[9]) % 2)
    .to.eql(p.gender === "female" ? 0 : 1);
});
```

Til bulk-requesten: assertér Array.isArray(data), data.length lig count og alle personers felter. Brug datadrevet kørsel til gyldige/ugyldige antal. Et manuelt klik i Send uden assertions opfylder ikke scriptkravet.

End-to-end med Playwright

Placér tests i PersonalData.E2ETests. Mindst én test skal gå gennem rigtig browser, frontend, API og MySQL-testdatabase uden at mocke API'et.

- E01 - Generér én hel person
  E2E | BB. Åbn forsiden, vælg en person og klik Generér data. Forvent ét kort med alle oplysninger og sammenhængende CPR, dato og køn.
- E02 - Generér flere personer
  E2E | BB: grænseværdier. Vælg Flere personer. Antalsfeltet vises. Indtast 2 og derefter 100; hver generering viser præcis det ønskede antal kort og erstatter tidligere resultater.
- E03 - Alle delvise valg
  E2E | BB: ækvivalensklasser. Afprøv navn/køn, navn/køn/dato, CPR, CPR/navn/køn, CPR/navn/køn/dato, adresse og telefon. Kun relevante felter vises; ingen undefined.
- E04 - Forkert antal i formularen
  UI med Playwright | BB. Indtast 1, 101 og tomt felt. Browserens validering stopper submit. Test direkte HTTP-omgåelse separat i A10; frontendvalidering er ikke nok.
- E05 - Fejl og efterfølgende succes
  UI med Playwright | ER + WB. Simulér HTTP 500 og netværksfejl. Forvent en læsbar fejl, ingen gamle kort og en genaktiveret knap. Næste succes fjerner fejlen og viser nye data.
- E06 - Ventetilstand og gentagne klik
  UI med Playwright | ER + WB. Forsink svaret kontrolleret. Knappen er deaktiveret, Genererer vises, og aria-busy er true. Efter svaret bliver siden klar igen; et ekstra klik under ventetid sender ikke en ny request.
- E07 - Danske tegn og lille skærm
  E2E + manuel udforskning | ER. Brug et kendt testdatasæt med æøåÆØÅ. Teksten gengives korrekt. Kontrollér desuden fx 390 px bredde, lange vejnavne og betjening med tastatur.

Sådan holdes browser-testene stabile

- Brug labels og roller som selectors: label for valg/antal og knappen Generér data. Vent på et synligt resultat eller en netværkshændelse; undgå faste sleeps.
- Kør E01/E02 mod rigtigt API og testdatabase. E05/E06 må gerne bruge Playwright-routing til fejl/forsinkelse; de er UI-tests og erstatter ikke rigtig E2E.
- Gem trace og screenshot ved fejl. Assertér regler og antal; forvent ikke et bestemt tilfældigt navn eller at to tilfældige personer altid er forskellige.

White-box og kodeanalyse

Dækning er dynamisk white-box-analyse. SonarQube er statisk kodeanalyse. De dækker forskellige krav og skal begge indgå i CI.

Brug Coverlet til at finde manglende grene

- Kør tests med linje- og grendækning. Find udækkede grene i rapporten, vælg ekstra testinput og kør igen. Gem før/efter-rapport som dokumentation for værktøjsbaseret analyse.

Kodegren	Kontrollerede input til Random / metode
NameGenderGenerator / AddressGenerator	Tom og ikke-tom liste; første og sidste indeks.
CprGenerator switch	Female, Male og ukendt enum; alle tre grene.
AddressGenerator: bogstav/etage	Next(100): 19/20 for husbogstav; 29/30 for st.
GetRandomDoor	doorType: 1, 7, 8, 14, 15, 16, 17, 18, 19 og 20.
PhoneNumberGenerator / fødselsdato	Præfikslængde 1/2/3; ekstra cifre 0/9. Dato-offset 0/sidste dag.
FakePersonService.GetPersons	-1, 0, 1, 2, 3, 99, 100, 101; begge fejlårsager og løkkens exit.

En fast seed gør kørslen gentagelig, men garanterer ikke alle grene. Brug fx en lille Random-subklasse med en kontrolleret sekvens til de relevante Next-overloads. Test private adressehjælpere gennem Generate().

Måling og statisk forbedring

```
dotnet test tests/PersonalData.UnitTests --collect:"XPlat Code Coverage" \
  -- DataCollectionRunSettings.DataCollectors.DataCollector.Configuration.Format=opencover
```

Kommandoen er vist med Bash-linjefortsættelse. I andre shells kan den samles til én linje uden backslash. Coverlet.collector er allerede i unit- og integrationstestprojekterne. Generér tilsvarende rapport for integrationstests.

- S01 - SonarQube: scan C# og JavaScript. Gennemgå pålidelighed, vedligeholdelse og sikkerhed; ret relevante fund og kør analysen igen. En linter alene opfylder ikke kravet.
- S02 - rapport og mål: importer coverage.opencover.xml med sonar.cs.opencover.reportsPaths. Aftal en quality gate; fx 80 % linjedækning og alle kritiske forretningsgrene er et forslag, ikke et opgavekrav.
- S03 - bevis for forbedring: gem fund, begrundelser, kodeændringer og ny analyse. Høj dækning alene beviser ikke korrekte assertions. Antag ikke præcise tilfældighedsprocenter i små tests.

CI og dokumentation

Foreslået værktøjskæde: GitHub Actions + xUnit + Coverlet + Postman/Newman + Playwright + SonarQube. Pipelinefilen skal implementeres; denne PDF er testdesignet.

Kør automatisk ved push og pull request

- C01 - opsætning: checkout, .NET 10, Node.js og værktøjer. Start en isoleret MySQL-testdatabase og indlæs addresses.sql med korrekt tegnsæt. Vent på healthcheck.
- C02 - begynd analysen og byg: start SonarScanner for .NET før build. Kør restore og build; angiv stier til de coverage-rapporter, som tests producerer.
- C03 - C#-tests: kør alle unit- og integrationstests med TRX-resultater og Coverlet. Hold DB-fejltests isoleret fra den database, andre tests bruger.
- C04 - API-tests: start API med testens connection string og vent på HTTP-readiness. Kør den eksporterede Postman v2.1-samling med Newman mod dette API.
- C05 - E2E: installér Playwright-browser og OS-afhængigheder. Kør alle tests i PersonalData.E2ETests mod den startede frontend og API.
- C06 - afslut analysen: upload scannerresultater og coverage til en SonarQube-server, som CI-runneren kan nå. Vent på quality gate, så et afvist resultat får jobbet til at fejle.
- C07 - håndhæv resultatet: fejlede tests eller afvist quality gate skal give rød pipeline. Gem TRX, coverage, Newman-rapport, Playwright-traces og analysebeviser; ryd processer og testdatabase op, også ved fejl.

Dokumentér jeres design og resultater

Lav en testcaseliste med ID, krav, testtype, designteknik, input, forventet resultat, faktisk resultat og bestået/fejlet. Brug ID'erne U/I/A/E ovenfor. Vedlæg manuel black-box-analyse, white-box-rapport før/efter, udforskningsnoter, rettede fejl og et CI-run med alle testtrin.

Officielle vejledninger til implementeringen

Coverlet: VSTest og rapportformater

Postman: Newman i CI (collection v2.1)

Playwright .NET: Continuous Integration

SonarQube: .NET-testdækning og scannerforløb